# CE 400/500

# Special Topics: AI for Chemical Engineers

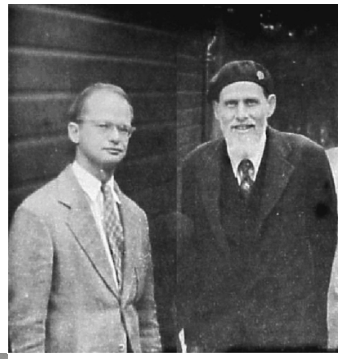
## Lecture 10: Introduction to Neural Networks

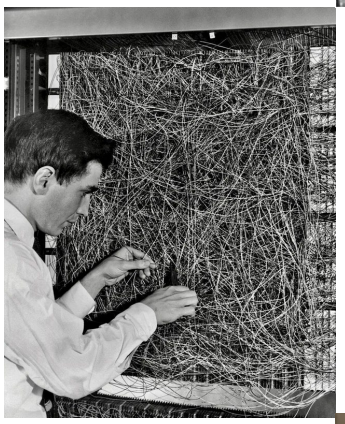
University at Buffalo
The State University of New York

# Brief History of Neural Network

**1943:** McCulloch & Pitts propose the first artificial neuron model.
**1958:** Rosenblatt introduces the Perceptron, an early learning algorithm.
**1969:** Minsky & Papert show perceptron limits → "AI Winter."
**1986:** Backpropagation popularized (Rumelhart, Hinton, Williams), revives interest.
**1990s:** Neural nets applied in speech recognition, handwritten digit recognition.
**2012:** AlexNet wins ImageNet competition with CNNs → major breakthrough in computer vision.
**2014 – 2017:** Advancements: GANs, sequence models (LSTM), attention & Transformers.
**2018 – :** Large language models (BERT, GPT, etc.) dominate AI research & applications.

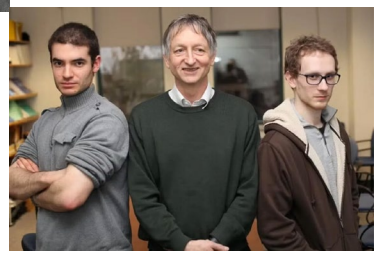
In 1943, at the University of Chicago, Warren McCulloch and Walter Pitts introduced the first mathematical model of a neural network





Frank Rosenblatt with a Mark I Perceptron computer in 1960

University of Toronto professor Geoffrey Hinton, (center), and graduate students Ilya Sutskever, left, and Alex Krizhevsky





https://machinelearning.uchicago.edu/history/

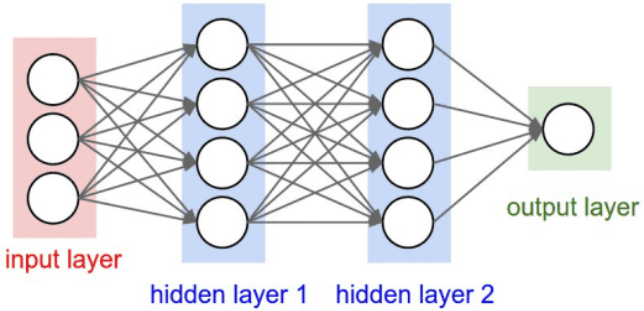
# Overview of Neural Network

**Neural networks** are computational models inspired by the brain, made of interconnected layers of neurons. They learn by adjusting weights and biases through training, enabling them to capture complex, **nonlinear** relationships in data.

**Architecture:**
- Input layer: receives raw data
- Hidden layers: extract patterns and features
- Output layer: produces predictions

**Building Blocks:**
- Neurons (nodes): fundamental computing units
- Weights & biases: adjustable parameters controlling signal strength
- Activation functions: introduce nonlinearity

**Training Mechanism:**
- Loss function: measures prediction error
- Backpropagation: updates weights



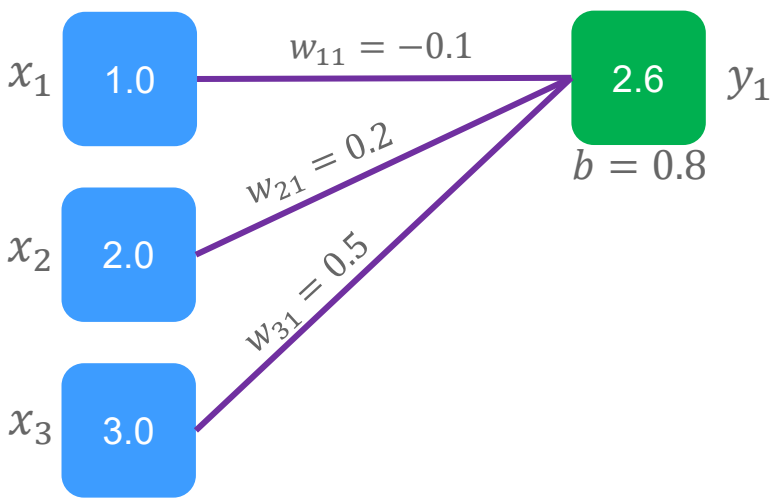

**Multilayer Perceptron**
(Feed-forward neural network, nonlinear regression)

# Nodes and Hidden Layers

Let's begin with the following familiar model structure: a linear model of the form:

$$y_1 = w_{11}x_1 + w_{21}x_2 + w_{31}x_3 + b$$

We can visualize this equation as shown below, where $x_1$, $x_2$, $x_3$ are our three input nodes (in blue), and $y'$ is our output node (in green).

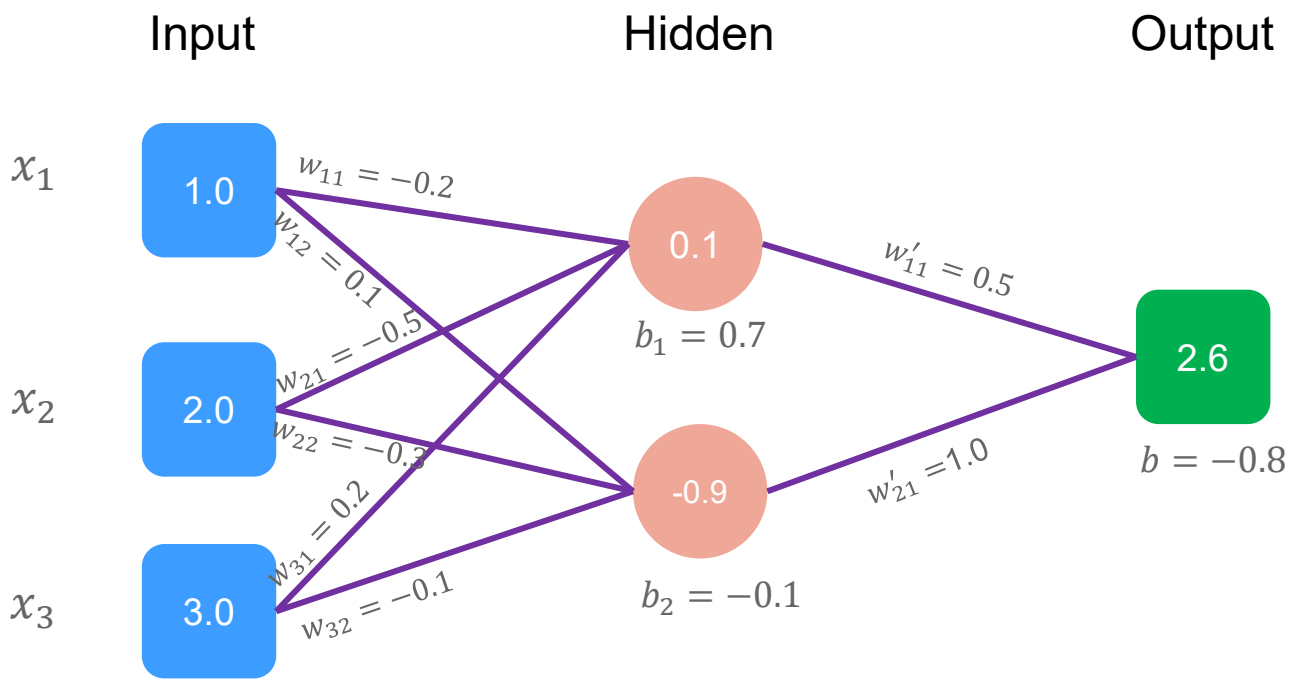



**Calculations**

= $w_{11}x_1 + w_{21}x_2 + w_{31}x_3 + b$
= -0.1 * 1 + 0.2 * 2 + 0.5 * 3 + 0.8
= -0.1 + 0.4 + 1.5 + 0.8
= 2.6

# Nodes and Hidden Layers (cont'd)

Now let's try to add a hidden layer containing two neurons to the model.




**Hidden node 1**
$= w_{11}x_1 + w_{21}x_2 + w_{31}x_3 + b_1$
= -0.2 * 1 - 0.5 * 2 + 0.2 * 3 + 0.7
= -0.2 - 1.0 + 0.6 + 0.7
= 0.1

**Hidden node 2**
$= w_{12}x_1 + w_{22}x_2 + w_{32}x_3 + b_2$
= 0.1 * 1 - 0.3 * 2 - 0.1 * 3 - 0.1
= 0.1 - 0.6 - 0.3 - 0.1
= -0.9

**Output**
$= w'_{11}x'_1 + w'_{21}x'_2 + b$
= 0.5 * 0.7 - 0.1 * 1 - 0.8
= 3.5 - 0.1 - 0.8
= 2.6

# Activation Functions

You saw in the previous exercise that just adding hidden layers to our network wasn't sufficient to represent nonlinearities. Linear operations performed on linear operations are still linear. **We can introduce nonlinearity by applying nonlinear transform of a neuron's output value via an <u>activation functions</u> before the value is passed as input to the calculations of the next layer of the neural net.**

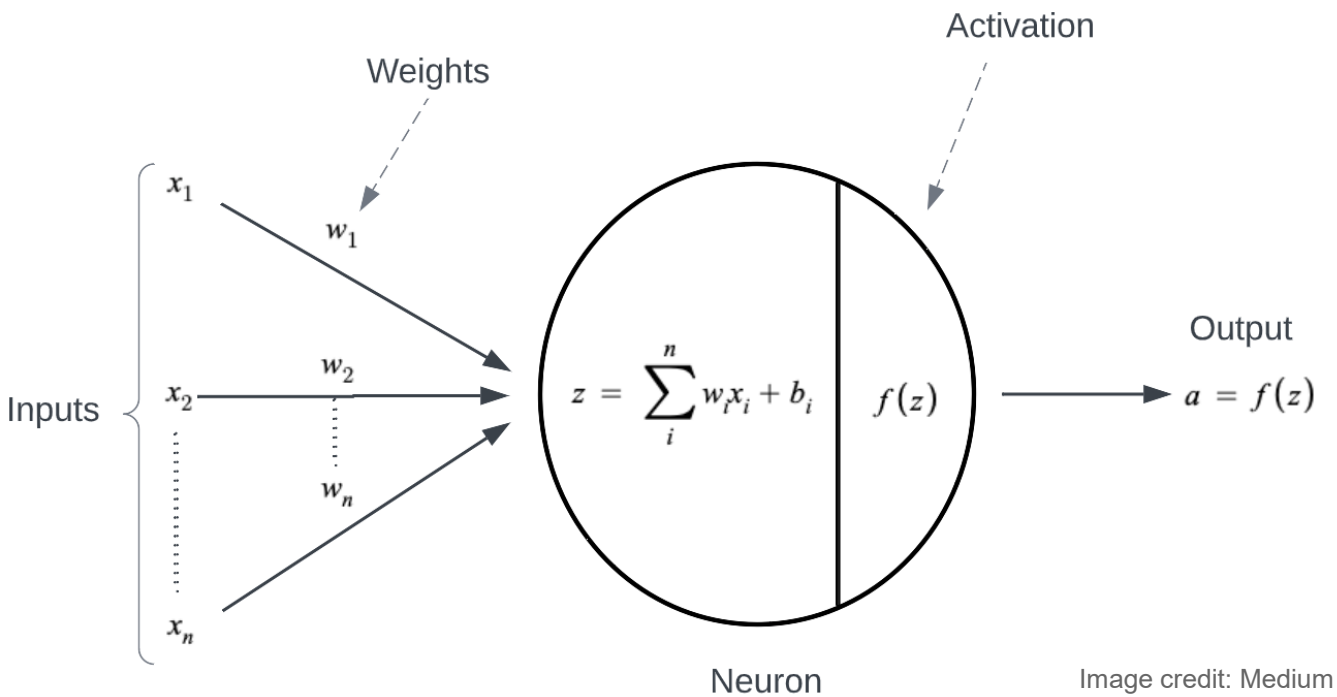
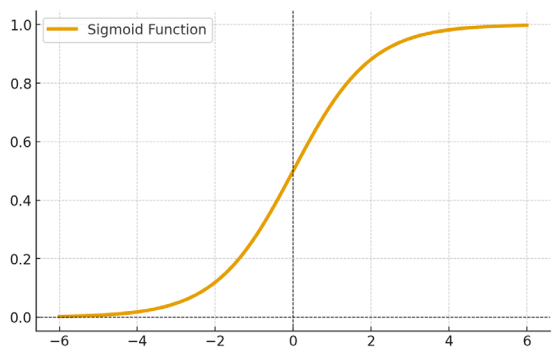



Image credit: Medium

# Common Activation Functions

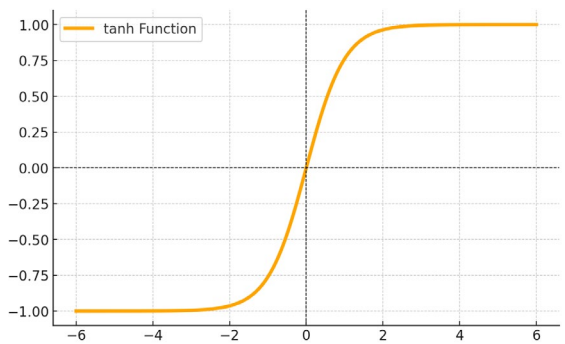
Three mathematical functions that are commonly used as activation functions are **sigmoid**, **tanh**, and **ReLU**.

The **sigmoid** function produces an output value between 0 and 1:



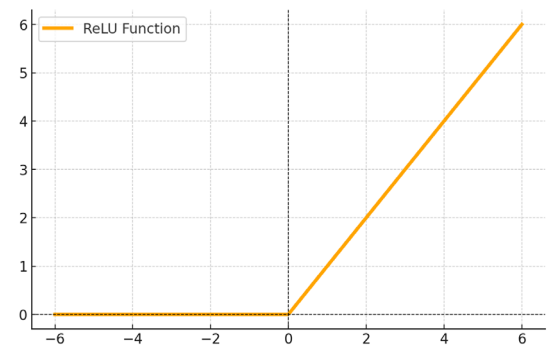

$$f(x) = \frac{1}{1 + e^{-x}}$$

The **tanh** (short for "hyperbolic tangent") function produce an output value between –1 and 1:




$$f(x) = \tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

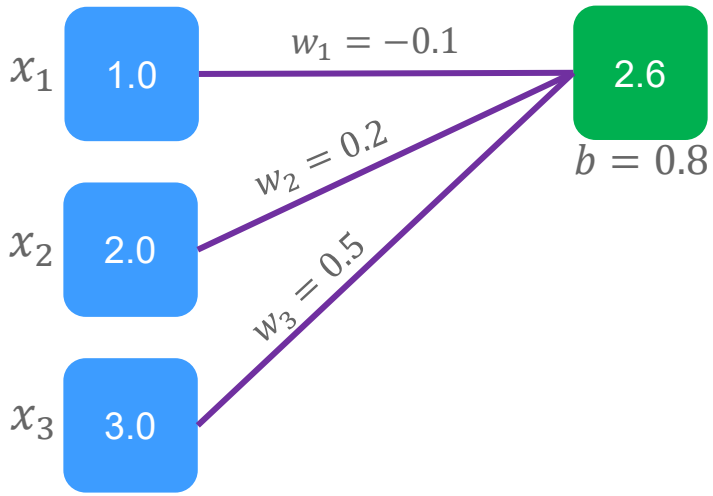
The rectified linear unit activation function (or **ReLU**, for short) transforms output using a piece-wise function:

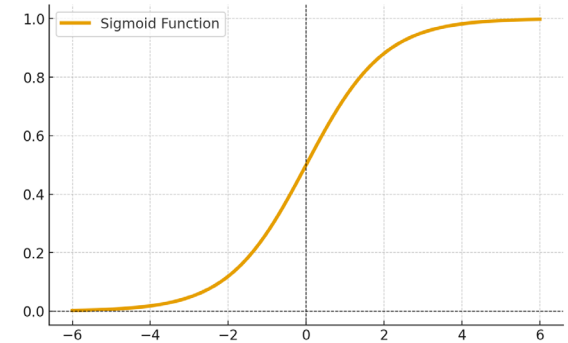



$$f(x) = \max(0, x)$$

# Activation Function Exercise

Original linear operation



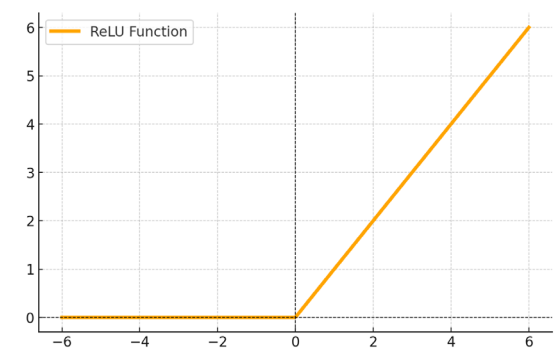




**Calculations:**

= $Sigmoid(w_1x_1 + w_2x_2 + w_3x_3 + b)$

= $Sigmoid(2.6)$

≈ 0.93




**Calculations:**

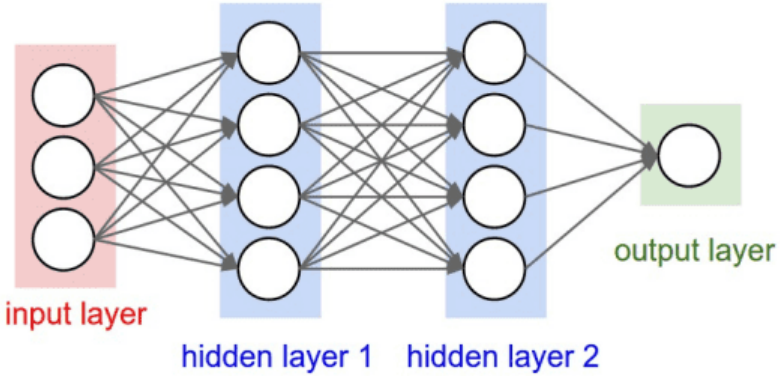
= $ReLU(2.6)$

= 2.6

# A Quick Summary

Now our model has all the standard components of what people usually mean when they refer to a neural network:

- A set of nodes, analogous to neurons, organized in layers.
- A set of learned weights and biases representing the connections between each neural network layer.
- An activation function that transforms the output of each node in a layer. Different layers may have different activation functions.



# Training Using Backpropagation

Backpropagation is the most common training algorithm for neural networks. It makes gradient descent feasible for multi-layer neural networks. Many machine learning code libraries (such as PyTorch) handle backpropagation automatically, so you don't need to perform any of the underlying calculations yourself.

We will use this interactive site to explain the backpropagation algorithm.

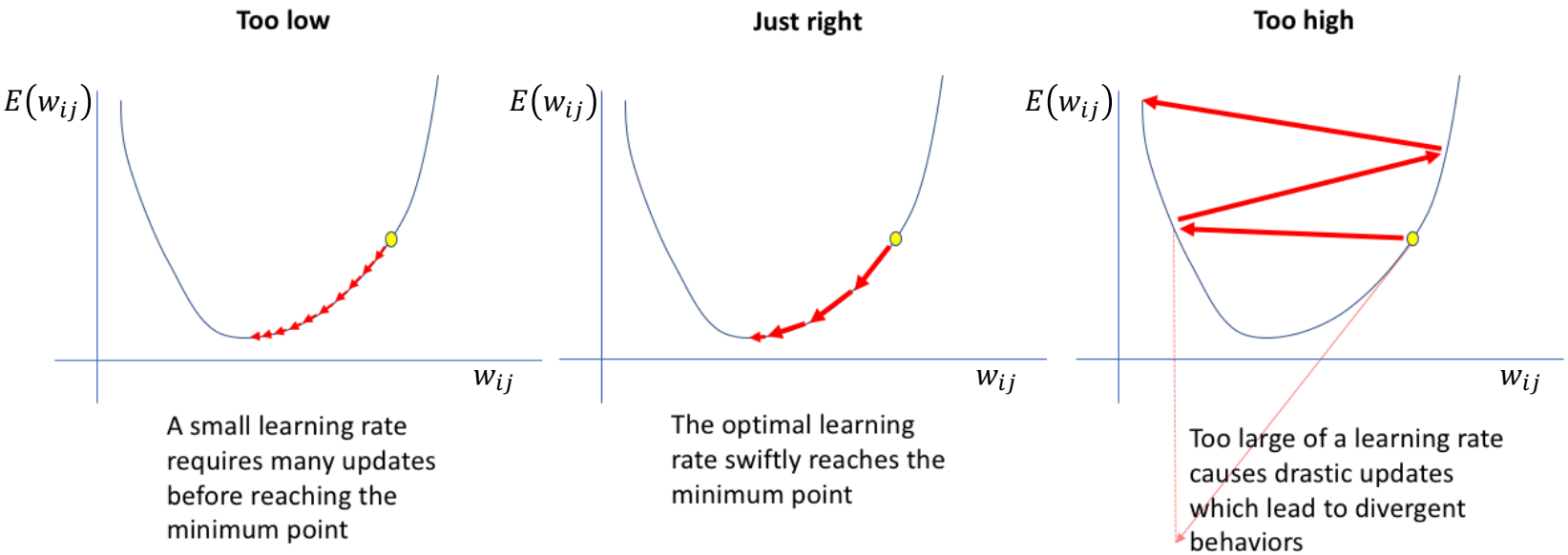


https://developers.google.com/machine-learning/intro-to-ml/supervised

# Learning Rate

$$w_{ij}^{(t+1)} = w_{ij}^{(t)} - \alpha \frac{dE}{dw_{ij}^{(t)}}$$

Learning rate (hyperparameter)




Adapted from the image by Jeremy Jordan

Let's try this interactive demo to learn the effect of learning rate.

- Pick "cos" dataset, try different learning rates, keep the same layer.

# Best Practices for Neural Network Training

**Vanishing Gradients**

The gradients for the lower neural network layers (those closer to the input layer) can become very small. In deep networks (networks with more than one hidden layer), computing these gradients can involve taking the product of many small terms. When the gradient values approach 0 for the lower layers, the gradients are said to "**vanish**". Layers with vanishing gradients train very slowly, or not at all.

**Solution: The ReLU activation function can help prevent vanishing gradients.**

$$w_{ij}^{(t+1)} = w_{ij}^{(t)} - \alpha \frac{dE}{dw_{ij}^{(t)}}$$

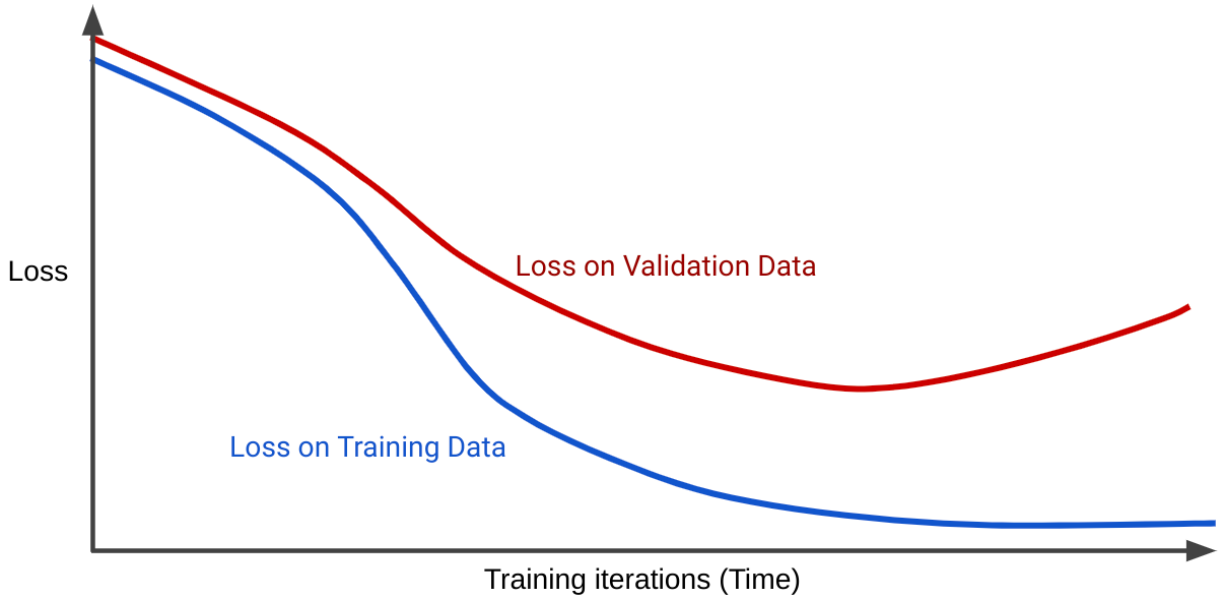
**Exploding Gradients**

If the weights in a network are very large, then the gradients for the lower layers involve products of many large terms. In this case you can have **exploding** gradients: gradients that get too large to converge.

**Solution: Lowering the learning rate can help prevent exploding gradients.**

# Detecting Overfitting for Neural Networks

A learning curve (or training curve) plots a model's loss against the number of training iterations (epochs). Notice that the two loss curves below behave similarly at first and then diverge. That is, after a certain number of iterations, loss declines or holds steady (converges) for the training set, but increases for the validation set. This suggests **overfitting**.

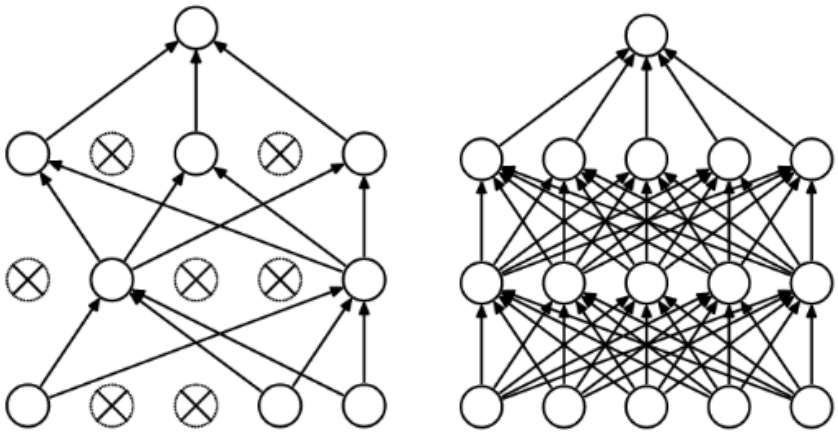



Reasons for overfitting:

- Model is too complex
- Training for too many iterations
- Training set is too small
- Presence of noise in data

# Methods to Overcome Overfitting

## Dropout layers



During training
After applying dropout

For final prediction
and inference

- Neurons are randomly dropped during training to reduce overfitting.
- At inference, all neurons are used with scaled outputs.
- The **dropout rate** sets the probability of dropping a neuron.

## Early stopping

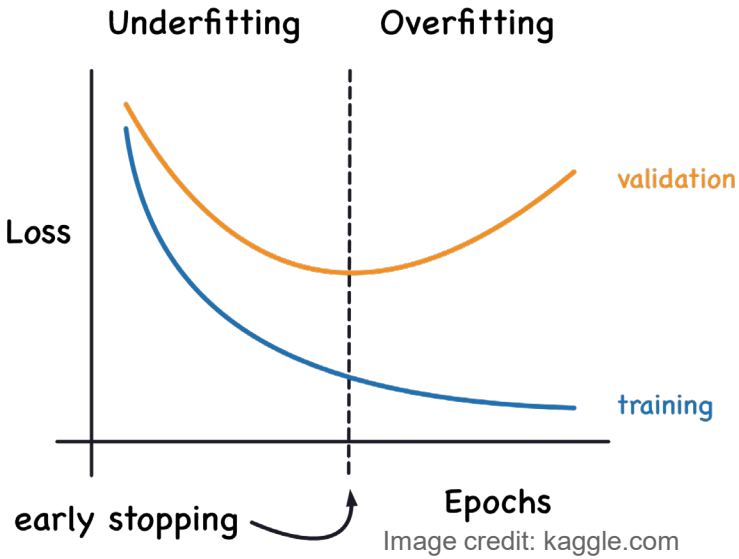
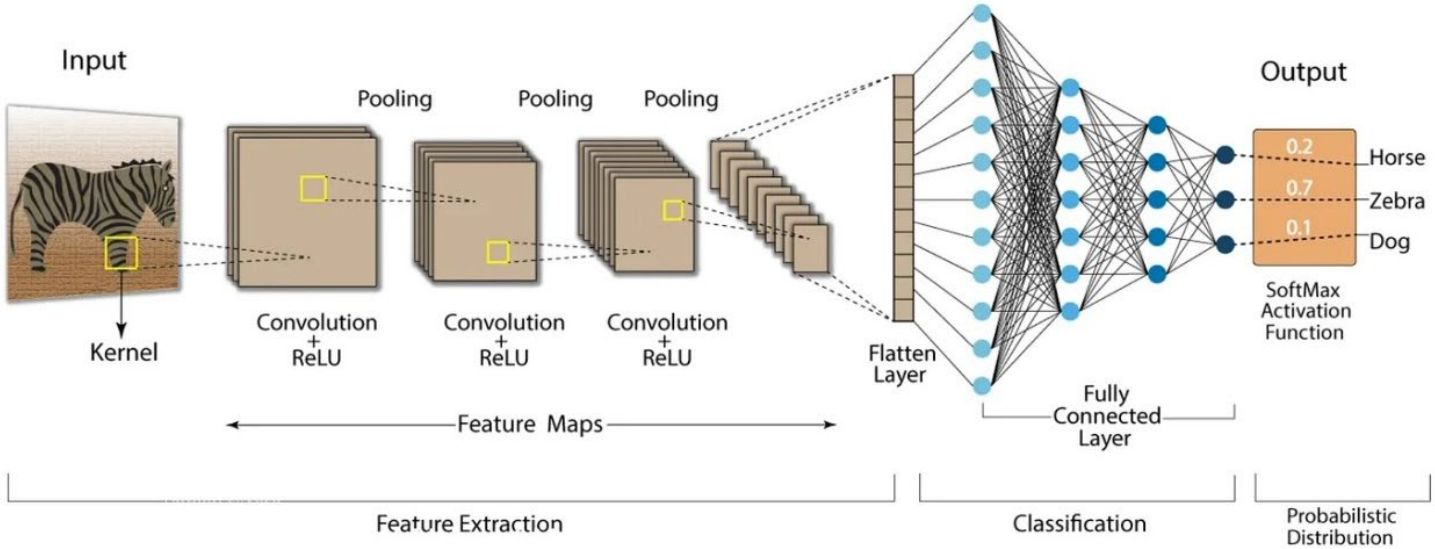
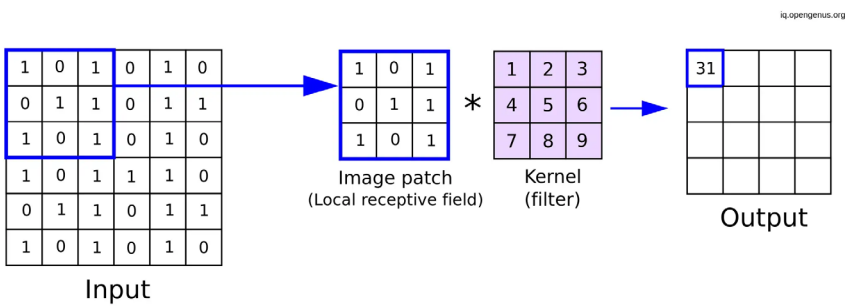



Image credit: kaggle.com

- Early stopping halts training when validation performance stops improving, preventing overfitting.
- Main parameter is **patience** – how many epochs to wait after no improvement.

# Advanced Architecture: Convolutional Neural Network



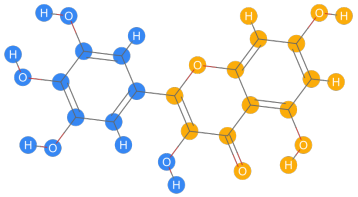

- Excellent for grid-like data (images, spectra, 2D/3D fields).
- Captures spatial/local correlations via filters via convolutional operations.
- Pooling layers reduce dimensionality while retaining important features.

- **Applications in chemical engineering**: Analyzing microscopy images (crystal morphology, catalyst particles). Spectroscopy data interpretation (Raman, IR). Predicting material properties from structural images / 3D atomic fields.

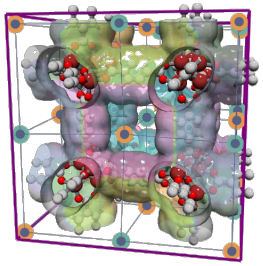


Image credit: Google

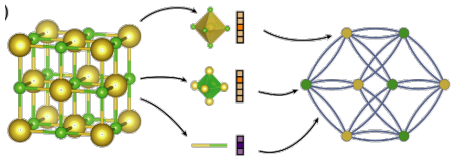
# Advanced Architecture: Graph Neural Network



**Molecular graph**
(image credit: Wolfram)

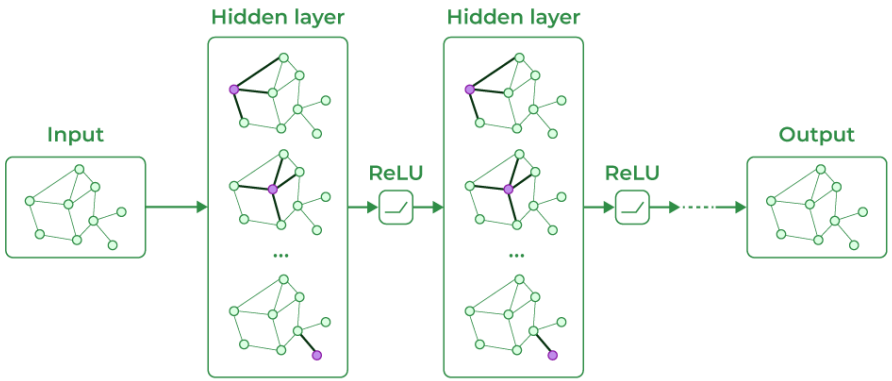


**Pore graph**
(developed by the Shi group)



**Crystal graph**
(Xie et al., Phys. Rev. Lett. 120(14), 145301 (2018))

**Knowledge graph, social graph and many more…**




**Graph convolution**
(image credit: GeeksforGeeks)

- Generalize the convolution operation of CNNs to graph-structured data (molecules, materials, pore networks).
- Captures relational/topological information.
- **Applications in chemical engineering**: Molecular property prediction (drug discovery, catalysts).Crystal/porous material design (MOFs, zeolites). Reaction network modeling. Knowledge graph enhanced LLM
- A Gentle Introduction to Graph Neural Networks

# Advanced Architecture: Recurrent Neural Network

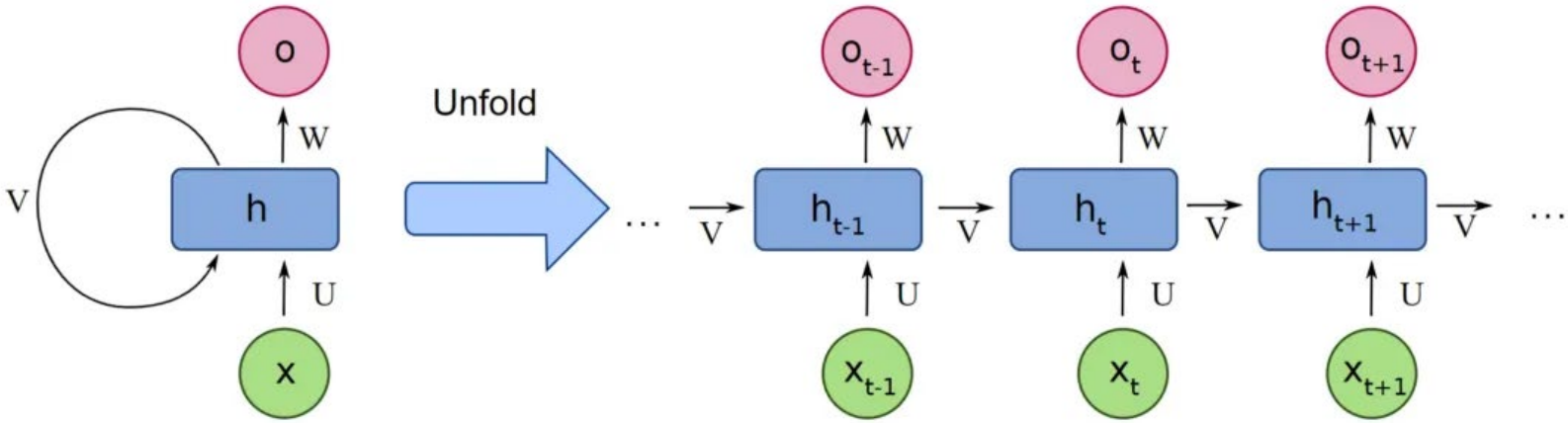



Image credit: Analytics Vidhya

- Designed for sequential/time-series data.
- Good when order and temporal dependence matter.
- Maintains hidden states that carry information from previous steps; can model dependencies across time.
- **Applications in chemical engineering**: Process monitoring and fault detection (plant sensor data). Predicting molecular sequences (e.g., polymer chains).

# Advanced Architecture: Residual Neural Network (ResNet)

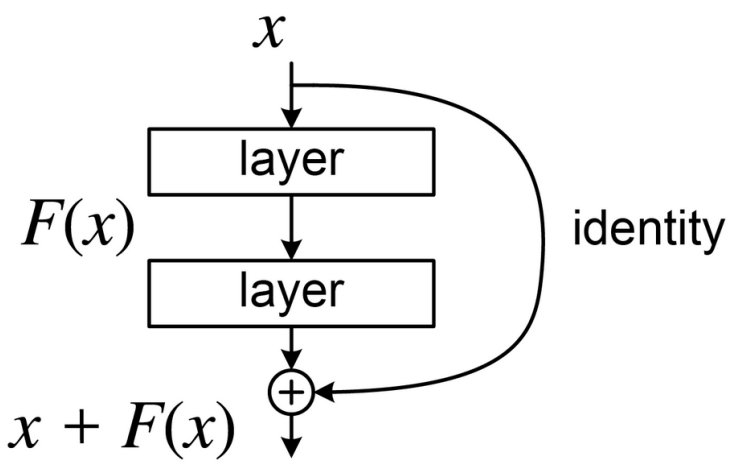



Image credit: Wikipedia

- Skip (residual) connections let very deep networks train without vanishing gradients.
- The skip/shortcut connection passes the input $x$ forward unchanged, so the layer only has to learn what is missing (the residual).
- **Applications**: High-resolution image/spectral analysis, microstructure and tomography in materials. Basically, ResNet shines when you need very deep feature extraction from images or spectral data.